

Project Lab

Pharmacy Management System

Developer Guide

Author:

Muhammad Hameez Khan

Supervisor :

AL-Magsoosi Husam Kareem Farhan

System Type	Desktop pharmacy operations and inventory system
Platform	Python, PyQt6, SQLite
Audience	Administrators, pharmacists, and software maintainers

1. Purpose

This guide explains how the Khalis Dawa Pharmacy Management System is organized, how it runs, and how developers can safely extend or maintain it. It is intended for coursework review, handover, maintenance, and onboarding.

Topic	Details
Application style	Desktop application with a multi-page graphical interface
Primary users	Administrators and pharmacists
Persistence	Local SQLite database file
Main goals	Inventory tracking, sales processing, prescription tracking, returns, and auditability

2. System Overview

The application provides end-to-end workflow support for a small pharmacy. The administrator or pharmacist signs in, navigates through dedicated pages, and performs operational tasks such as managing products, updating batches, selling medicines, recording prescription-required purchases, processing returns, and reviewing audit activity.

- Role-based access separates administrator responsibilities from pharmacist responsibilities.
- Batch-aware stock handling supports expiry tracking and FEFO-style allocation during sales.
- Prescription-required medicines trigger customer identity capture in the point-of-sale workflow.
- Returns restore stock, reverse revenue, and produce their own records and reports.
- Audit logs record meaningful actions for accountability.

3. Technology Stack

Layer	Implementation	Notes
Programming language	Python 3	Core application logic
GUI framework	PyQt6	Desktop user interface and dialogs
Database	SQLite	Local embedded database stored in database/pharmacy.db
Reporting	HTML exports	Sales receipts, return reports, and prescription reports
Project style	Single-application desktop project	Code organized by pages and database functions

4. High-Level Architecture

The project follows a straightforward desktop architecture. Presentation code lives in page modules, while data access and most business rules live in database/db.py.

- main.py initializes the database and launches the login window.
- login_window.py authenticates the user and opens the main shell.
- main_window.py builds the sidebar, top header, and stacked page navigation.
- pages/*.py files define feature-specific interfaces.
- database/db.py centralizes schema creation, CRUD operations, returns logic, prescriptions, sales, and audit logging.

5. Project Structure

Path	Responsibility
assets/	Static assets such as icons and visual resources
database/db.py	Database schema, queries, writes, business rules, and report data
database/seed_data.py	Optional sample data support
pages/dashboard.py	Dashboard statistics and summary tables
pages/batch_management.py	Batch CRUD, filters, and batch presentation
pages/pos.py	Point-of-sale workflow and customer capture dialogs
pages/product.py	Product catalog management
pages/prescriptions.py	Prescription tracking table and report downloads
pages/sales_history.py	Completed sales, receipts, and return entry point
pages/returned_items.py	Completed return records and return reports
pages/audit_logs.py	Audit activity review
pages/settings.py	Password management and administrative account updates
login_window.py	Authentication entry screen
main_window.py	Main application shell and role-based navigation
ui_theme.py	Shared constants and UI styling helpers

6. Database Design

The database is automatically initialized at startup. Key tables are summarized below.

Table	Purpose	Important fields
users	Authentication and authorization	username, password, role
audit_logs	Traceability for meaningful actions	username, role, description, created_at
products	Product catalog	name, dosage_form, barcode, selling_price, category, prescription required
batches	Stock by batch and expiry	product_id, batch_number, expiry_date, quantity, discount_percent
prescriptions	Prescription-linked sale records	sale_id, customer_ssn, customer_name, medicine_name, amount, created_at
sales	Sale header records	sale_datetime, total_amount, cashier_id,

		payment_method
sale_items	Per-product sale lines	sale_id, product_id, product_name, quantity, unit_price, subtotal
sale_batch_allocations	Batch allocations used for sale fulfillment	sale_item_id, batch_id, quantity, expiry_date
returns	Return header records	sale_id, processed_by_username, total_amount, return_datetime
return_items	Returned sale item quantities	return_id, sale_item_id, quantity, refund_amount
return_batch_allocations	Stock restoration by original batch	return_item_id, batch_id, quantity

7. Core Functional Workflows

7.1 Authentication and Roles

- The system seeds a default administrator account on first launch.
- Login authenticates by username and password against the users table.
- Administrators can access every page, including Settings.
- Pharmacists are restricted from selected destructive administrative actions.

7.2 Product Management

- Products store pricing, dosage form, barcode, category, minimum stock level, and whether a prescription is required.
- Duplicate name and barcode checks are enforced before inserts or updates.
- Product tables expose low-stock and available-stock context from related batch data.

7.3 Batch Management

- Batches track stock quantity and expiry date separately for each medicine.
- Expiry-sensitive logic supports low-stock monitoring and expiring-soon dashboards.
- Discounts are automatically applied to qualifying batches nearing expiry.

7.4 Point of Sale

- The POS page supports quantity selection before items are added to the cart.
- Sales are allocated using available non-expired batches.
- Prescription products require the customer SSN and customer name.
- Payment method is captured during checkout.
- Audit logs record the sale outcome.

7.5 Prescriptions and Reports

- Prescription records store customer SSN, customer name, medicine, quantity, and sale timestamp.
- Prescription reports are downloadable per record.
- The prescriptions page is for tracking sensitive medicine sales rather than general shopping activity.

7.6 Returns

- Returns are limited by a configured return window of three days.
- Returned stock is restored to the original batch allocation whenever possible.
- Revenue calculations subtract processed returns.
- Completed returns have their own records, downloadable reports, and audit entries.

7.7 Audit Logging

- The audit log focuses on meaningful state-changing actions.
- Examples include sign-in, product changes, batch changes, sales, returns, account creation, and password changes.
- Grouped sale and return messages are used to avoid noisy one-line-per-unit logs.

8. Setup and Execution

1. Install Python and PyQt6 in the environment used for the project.
2. Open a terminal in the project root.
3. Run the application with `python main.py`.
4. Log in with the default admin account on first launch and change the password after access is confirmed.

Default credential	Value
Username	admin
Password	1234

9. Page Responsibilities

Page	Primary purpose	Typical users
Dashboard	Operational overview and alerts	Admin, Pharmacist
Batch Management	Batch creation, update, expiry and stock review	Admin, Pharmacist
Point of Sale	Sales processing	Admin, Pharmacist
Product	Product catalog CRUD	Admin, Pharmacist
Prescriptions	Prescription sales review and report export	Admin, Pharmacist
Sales History	Receipt review and return initiation	Admin, Pharmacist
Returned Items	Processed return review and return report download	Admin, Pharmacist
Audit Logs	Traceability and oversight	Admin, Pharmacist
Settings	Password management	Admin only

10. Extension Guidance

- Prefer extending business rules in `database/db.py` rather than duplicating logic inside page files.
- Keep page modules focused on user interaction and presentation.
- If a new workflow changes stock, sales, returns, or security-sensitive behavior, add corresponding audit log coverage.

- When adding a new page, wire it through `main_window.py` and preserve role-based access rules.
- Use the existing report-generation pattern when introducing additional printable documents.

11. Troubleshooting

Issue	Likely cause	Recommended action
Login fails for expected user	Incorrect password or missing seeded account	Verify users table and default admin creation in <code>initialize_database()</code>
Prescription workflow blocks checkout	Prescription medicine is in cart without customer SSN/name	Provide required customer details or remove prescription-only products
Return cannot be processed	Sale is outside return window or item is no longer eligible	Review sale date, previous returns, and batch validity
Dashboard values look inconsistent	Underlying stock, return, or expiry data changed	Check batches, sales, and returns tables together
Receipt/report export missing	Downloads directory or export flow interrupted	Retry from the source page and verify generated file output

12. Known Limitations and Improvement Areas

- The project currently relies on a local SQLite database rather than multi-user network storage.
- The repository does not yet include an automated test suite.
- Dependencies are lightweight, but a formal `requirements.txt` would improve reproducibility.
- Backup/export and restore workflows would strengthen operational resilience.
- Packaging as a desktop installer or executable would improve deployment for non-technical users.